

Getting LLM to think and act like a human being: Logical path reasoning and Replanning

ABSTRACT

Large Language Models (LLMs) have a vast capacity for thinking and reasoning and can interact with the real world as agents, but they are segmented and lack integrated systems for validating thoughts and actions like humans. This limitation often results in LLMs falling into the trap of "Local Optima" in task performance. To address this issue, we introduce a Replanning mechanism for LLM-based agents that dynamically integrates feedback from actions and implicit information which not available in an initial thinking and reasoning framework, forming a bridge between the thinking and acting of LLMs. Compared to other agent methods, experiments conducted on real-life ticket booking websites such as Ctrip and Booking show that our method is more robust in executing clear instructions, capable of successfully completing more steps, and achieving a higher success rate in practical tasks such as ticket booking, especially in challenging tasks that require interactive thinking and action for LLMs.

CCS CONCEPTS

• **Do Not Use This Code → Generate the Correct Terms for Your Paper;** *Generate the Correct Terms for Your Paper;* Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

KEYWORDS

LLM, Agents, Replanning, Reaction, Logical path reasoning

ACM Reference Format:

. 2018. Getting LLM to think and act like a human being: Logical path reasoning and Replanning. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 12 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

During the three seminal industrial revolutions, the advent of the steam engine revolutionised the large-scale transportation of goods, the development of electrical power transformed the efficiency and convenience of energy distribution, the rise of computer technology radically streamlined the dissemination of information, and the artificial intelligence technologies represented by Large Language Model (LLMs) have now simplified the transfer of knowledge to an unprecedented level, leading the wave of the Fourth Industrial Revolution [2, 7, 16, 36].

And people have placed higher expectations on LLMs, hoping that they can solve more challenging problems. In this context, people are studying how to unlock the powerful thinking and reasoning capabilities of LLMs. The most basic method can be traced back to the use of prompts [11]. Prompts enable LLMs to form identity cognition and focus on completing specific scenarios and tasks, thus making more accurate judgments on simple questions. However, LLMs still perform mediocrely when faced with increasingly complex questions from people. To address this issue, the Chain of Thought (COT) [37] enables LLMs to engage in deeper understanding and reasoning step by step before directly outputting an answer. This improves their effectiveness in solving single problems compared to simple prompts. However, even with COT, complex multi-step problems cannot always be solved with a single output. The planning approach [25] involves decomposing complex multi-step problems into sub-problems, which enhances reasoning capabilities. Various methods aim to improve the thinking ability of LLMs and obtain more accurate answers.

However without the ability to interact with the real world, LLM is merely like a brain in a vat [20, 23]. Giving LLMs the ability to interact with the real world is the first key towards general artificial intelligence (AGI) [5]. This capability is based on treating LLMs as agents, teaching them to think and reason like humans, and acting on the basis of these cognitive processes [14, 17, 26]. Unfortunately, such studies generally perceive the thinking and acting of LLMs as a segmented process [39]: First, thinking and reasoning are used to formulate a plan based on existing information, which is then carried out as an action in a procedural manner. For example, if a LLM agent needs to book an airline ticket for someone online [44], it can easily formulate a plan such as logging on to the website, selecting the destination of the flight, the place of departure, the time, and so on. It then follows the plan to complete the specific actions. However, if, after selecting the flight details, the task involves selecting a cabin class, the LLM may not provide a correct solution and thus fail the task, as this was not mentioned in the initial plan.

In reality, human thought and action patterns represent an interactive process in which planning and action guide and correct each other [31]. During this process, people continuously perform actions based on their plans to explore the real world. More importantly, they integrate newly acquired information from this exploration with previously unconsidered implicit information [22], thereby adjusting their plans and action strategies. This implicit information comes from people's deep reasoning about scattered pieces of task-relevant information accumulated in reality. Even after completing a task, people can learn more general and universal knowledge through deduction and induction from this experience, it's mankind's most valuable asset. Using the example of booking a flight, when faced with such a situation, humans would reason based on the specific identity and background of the person giving the instruction, such as their financial status or whether the

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted by ACM, provided that the copies are made without charge and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, June 03–05, 2018, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/18/06
<https://doi.org/XXXXXXX.XXXXXXX>

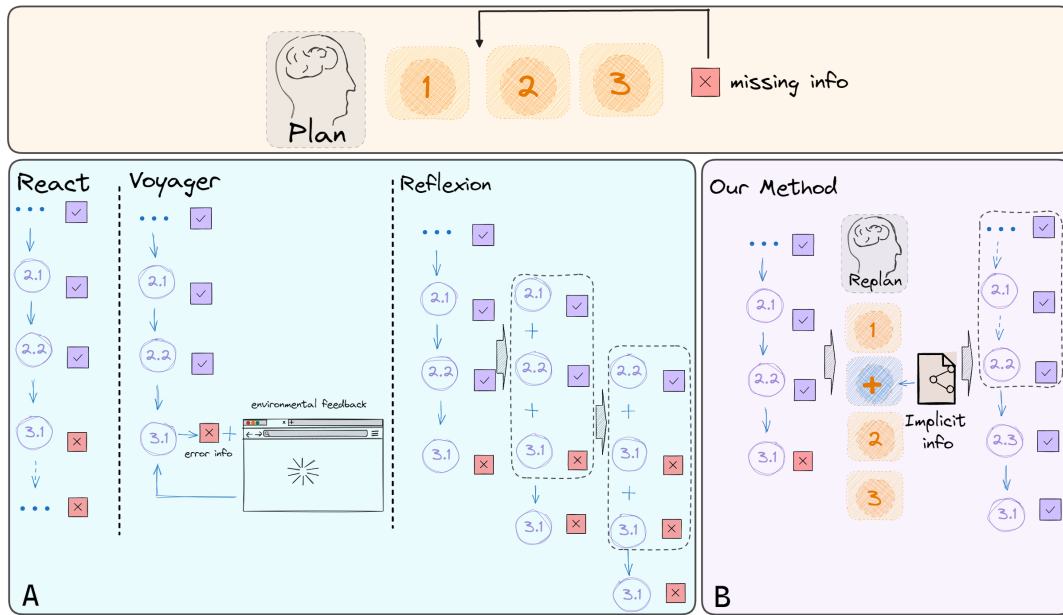


Figure 1: Comparison of existing LLM agents methods with our method: The top shows LLM’s reasoning and plan, below is how each method deals with error caused by the Initial plan.

trip was on official business. These disparate pieces of information would be taken into account in the planning stage to decide which type of cabin to choose, and thus proceed smoothly to the next step. In future instances of booking a flight, the action of selecting a cabin class would be incorporated into the planning phase.

Existing agent methods fail to fully integrate the behaviour of Large Language Models (LLMs) into a unified process of reasoning and action. [13] This limitation prevents LLMs from rethinking based on global information with implicit information, especially when they encounter errors. They can only correct their actions based on error messages about the current state. In attempting to revise their plans, they do not proactively acquire implicit information that was not previously considered. Instead, they simply iterate and correct errors by putting all action information and feedback received into the LLM’s prompt, thereby falling into the trap of "local optima" in task performance. For example(Fig. 1A), in the React method [42], LLMs progressively receive external feedback and base their thinking and actions on the current information, thus initiating cycles of feedback, thinking and action. Action in the current step is based solely on thinking about the feedback for that step. If a problem arises at an intermediate step, such as the lack of cabin class information in the example, it will continue with the missing information and get stuck. Voyager [33] and Reflexion [24] are two methods that can partially address these shortcomings. Voyager considers both feedback from the current environment and error information to reconstruct the current action. Reflexion uses the entire process of one or more steps as feedback for correction, to some extent taking into account a broader set of information. With these method, agents may infer from the web page in the example and the previous processes that a cabin class selection is

now required [40]. However, it still lacks the implicit information needed to further modify the plan and make a rational choice.

To address this problem, we have introduced a Replanning mechanism for LLM agent, endowing them with human-like cognitive processes and dynamic interactive capabilities using the real-world scenario of online ticket booking as an example. This mechanism dynamically integrates experience from action feedback and infers task-relevant implicit information from scattered external knowledge bases(Fig. 1B). This enables LLM agent to correct erroneous actions by Reaction based on this information, then re-evaluate and adapt to refine more viable plans, and ultimately derive more general action strategies. It utilizes three innovative mechanisms(Fig. 2): implicit reasoning, Replanning, and Reaction, making the following three major contributions:

- Implicit reasoning based on logical path propagation:** We spread the explicit information in the text along the logical paths formed by entities and their interrelationships, thereby generating a coherent and in-depth reasoning structure, integrating implicit information for reasoning. This process enables large language models to construct a mind map integrating explicit and implicit information, achieving a cognitive process closer to human-like thinking.
- Replanning:** A Replanning mechanism based on action feedback is proposed. This allows planning to dynamically integrate experiences from successful or failed actions, linking these isolated local optimal decisions for globally optimal Replanning. This flexibility enables the model to adapt to changing demands and complex environments, reducing planning illusions and increasing the accuracy of correct thinking and action.

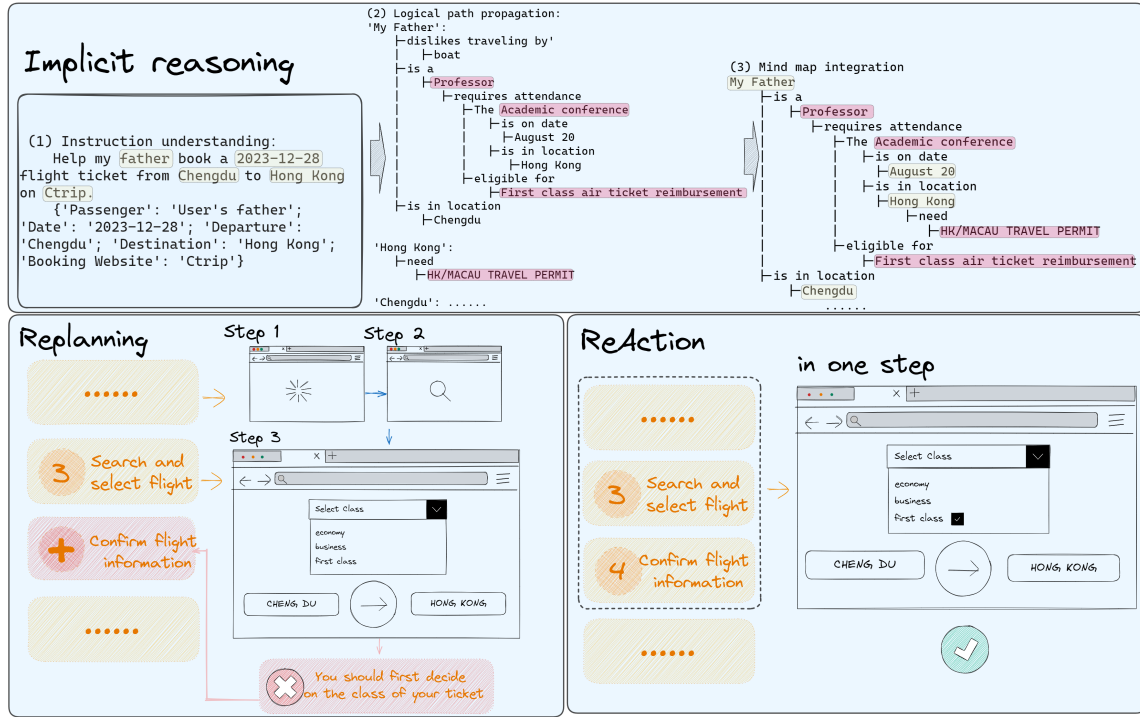


Figure 2: Implicit reasoning and Replanning using a real-life example of online ticket booking.

- **Reaction and inductive deduction based on Replanning:** Based on Replanning, we proposed the Reaction and inductive deduction mechanism. This not only corrects erroneous actions and explores new actions but also reassesses and corrects action strategies under a global information set. More importantly, throughout this dynamic Replanning and reaction interaction process, more general laws and strategies are refined and preserved. When facing new, complex, and interrelated tasks, these principles and experiences are quickly utilized. This enhances the efficiency and accuracy of thinking and actions and strengthens adaptability and generalization in the face of unknown and novel situations.

2 METHOD

This paper discusses the thinking and action paradigm of the Large Language Model (LLM) based on implicit expression and Replanning. The capabilities are primarily contributed by three new components: (1) logic path reasoning based on KG (Sec 2.1); (2) single action process incorporating environmental feedback and skill management (Sec 2.2); (3) task chain management for implementing Replanning and Reaction (Sec 2.3).

2.1 Logic path reasoning based on KG

To capture implicit information based on the identity, background, and preferences of the user, we manage discrete and sparse static user data in a vector database in the form of Knowledge Graphs (KG). The KG are established by triples of entities and relations, [head, relation, tail], and offer the following advantages:

- **Symbolic Reasoning:** Symbolic Reasoning: KG provides an objective representation of human intuition and uses clearly defined symbols and rules to explicitly illustrate the comprehensive logic and relationships of additional knowledge [8].
- **Dense Information:** KG allows for the efficient storage and processing of large amounts of user data, resulting in faster responses during extended tasks, such as booking tickets [35].
- **Path Propagation:** There are many feasible methods to connect KGs to form a complete path [38]. Such a convenient path propagation mechanism enables the identification of relevant entities and the development of coherent logical pathways, leading to the acquisition of new knowledge.

In our research paradigm, reasoning based on Knowledge Graphs (KG) follows a logical path that involves the following key steps (Fig. 3):

- (1) **Named entity recognition and KG retrieval:** LLM identifies the core entities in the user-issued ticket booking instructions. This step forms the foundation of the entire knowledge retrieval process, ensuring accuracy in subsequent operations.
- (2) **Path Propagation:** Once entities have been identified, they are matched with a KG vector database containing basic facts and user information. The retrieved KGs propagate along the paths between entities and relations, resulting in multiple logical paths. Each identified entity acts as the **head** in the KG knowledge triples, finding each **tail** entity associated with the **head** entity with the **relation**. The path is iteratively expanded

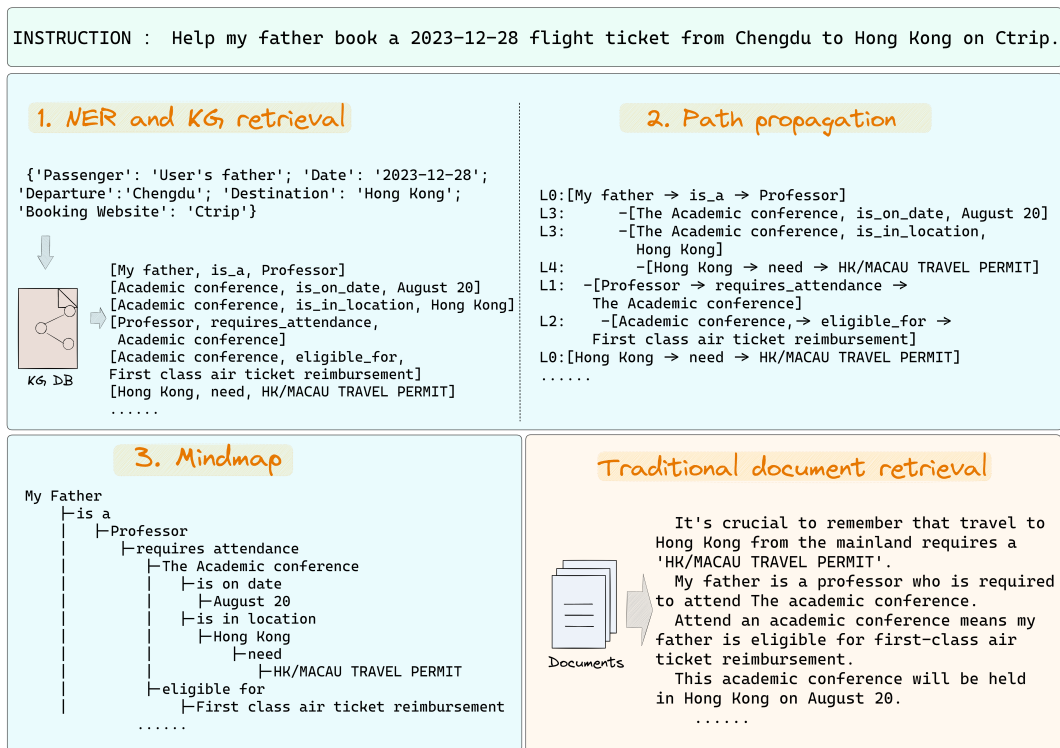


Figure 3: Steps of Logic path reasoning based on KG and the difference with traditional document retrieval.

by making the **tail** entity the new **head** entity until no further **tail** entities are found or until entities repeat.

(3) **Mindmap generation:** To generate a complete KG mindmap, use the path propagation results to determine the complete logical path for each initial instruction entity. To adapt the information to the LLM’s symbolic reasoning capabilities, we merge logical paths based on entity relationships and remove irrelevant information to create a comprehensive KG Mindmap. The Mindmap includes both the information mentioned in the user instructions and implicit knowledge obtained through path searches, which serves as the foundation for creating ticket booking plans.

This approach achieves an expression closer to human cognitive processes and transforms sparse, document-based knowledge repositories into a more effective knowledge graph-based reasoning and rapid response mechanism.

2.2 A single action process based on environmental feedback and skills management

To enable the interaction of Replanning and Reaction in LLM’s actions, LLM’s single action must be manageable, generalisable, and adjustable throughout the planning process. [15, 21] We treat LLM’s single action as a dynamic skill management system based on environmental feedback. For example, when a flight booking task

is executed up to the stage of determining the origin, destination and time. The process is as follows(Fig. 4):

- (1) **Environmental Awareness:** Real-time web environment information, including web text, interactive elements, and corresponding HTML, is integrated into the dynamic action process of LLM. The cleaned web page text and interactive components are used as inputs to LLM, which then selects the origin, destination, and time to be triggered.
- (2) **Skill library management:** Once the current task and interactive elements have been clarified, LLM queries a skill library to check for directly applicable skills. This library is managed by a vector database that embeds vector keys describing the skill's applicable scenarios and input information. The values are Python Selenium scripts for triggering web operations. If the relevant skill exists, LLM will execute the script, receive real-time web information as feedback for the next action.
- (3) **Error handling mechanism based on environmental feedback:** Section three introduces a global error handling mechanism that involves Replanning and Reaction strategies based on task chain management. Furthermore, to prevent random errors during the single skill creation process, such as errors in code generation, an error handling mechanism based on environmental feedback has been established. When an error is encountered, the code interpreter provides feedback, and the environmental awareness mechanism is re-invoked to check

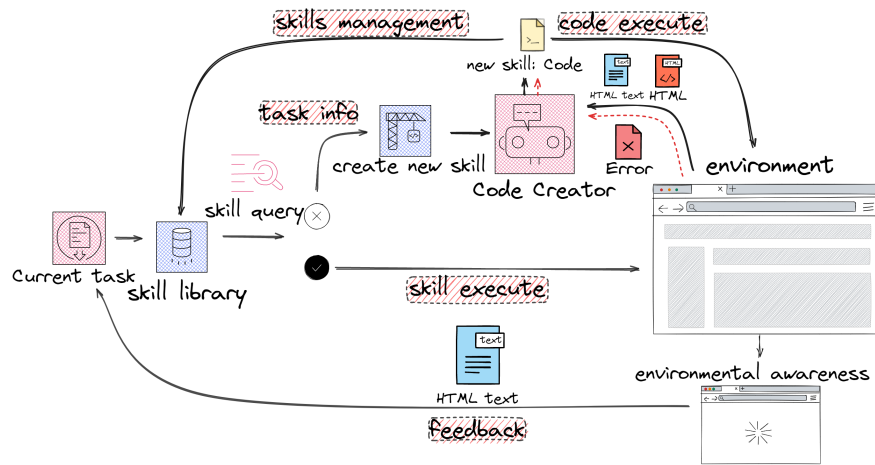


Figure 4: Steps of single action process based on environmental feedback and skills management.

and proceed with another round of skill creation. This iterative process occurs twice.

Through this design, our actions are robust and gradually form a generalisable library of skills. Each action is recorded as a KG triplet [*current task*, *action execution state*, *next task*] to serve as a basis for Replanning and Reaction in Sec2.3.

2.3 Task chain management

2.3.1 Replanning. Although we’ve designed an iterative error-handling process for single actions in Section 2.2, when errors occur due to lack of information, it’s essential to introduce new information on a global scale to reconstruct the plan. For example, if the original plan did not take into account the selection of aircraft seats, and the task fails after two iterations on the current page, a Replanning mechanism is introduced as follows(Fig. 5):

- (1) **Task chain generation:** Concatenate the [*current task*, *action execution state*, *next task*] triplets from the start to the point of failure into a single task chain. The *action execution status* includes the skill called or created, input values, success status and feedback received. This task chain not only provides a complete process flow, but also has the ability to integrate multiple actions into a single sequence of operations from top to bottom.
- (2) **Replanning:** Replanning the single task chain by introducing implicit information enables the LLM to identify missing information from the task chain and create a workable plan. This allows corrections and resumption of the task chain from the point of interruption.

This approach ensures that the LLM can adapt and respond flexibly and efficiently in complex and dynamically changing task environments, ensuring successful task completion.

2.3.2 Reaction. Based on the task chain linked by the current action triplets, which records callable skills in the action execution state, we essentially memorise a directly executable and feasible action path. When correcting errors, we can trigger all feasible

actions sequentially, thereby integrating the task chain and starting from the beginning with LLM reasoning for each action.

We also use a vector database to store different task chains. These chains are merged based on the vector similarity of the tasks, creating a comprehensive mindmap that encompasses multiple task chains. This mindmap dynamically integrates and manages actions triggered in the past, following the logical relationships of the planned tasks. When similar or identical tasks need to be performed again, there’s no need to rebuild the plan or call skills for each action. Path matching quickly retrieves similar or identical chains of tasks, and by using the Reaction method, multiple steps are executed at once. This approach not only reduces the likelihood of random errors in complex tasks involving multiple interactions, but also significantly reduces the time and token consumption required for LLM reasoning. This method increases the efficiency and accuracy of task execution, enabling the LLM to handle complex tasks more intelligently and effectively, reducing error rates and processing time, and improving overall system performance.

3 EXPERIMENTS

In this section we aim to answer three main questions:

1. Compared to other baseline methods, is our Replanning-based method the best at completing tasks on real-world ticket-booking websites?
2. Does the agent based on Replanning and Logical path reasoning still perform well on complex, challenging tasks that require effective combination of implicit information like a human, where initial instructions are unclear and the task chain is longer?
3. How do these different design choices (Replanning and response, Logical path reasoning, and Reaction) in our method affect performance of the system?

3.1 Experimental Setup

We used OpenAI’s gpt-4 [2] and gpt-3.5-turbo-0613 [9] APIs for text completion, and the text-embedding-ada-002 API for text embedding [10]. We set all temperatures to 0. In order to simulate the

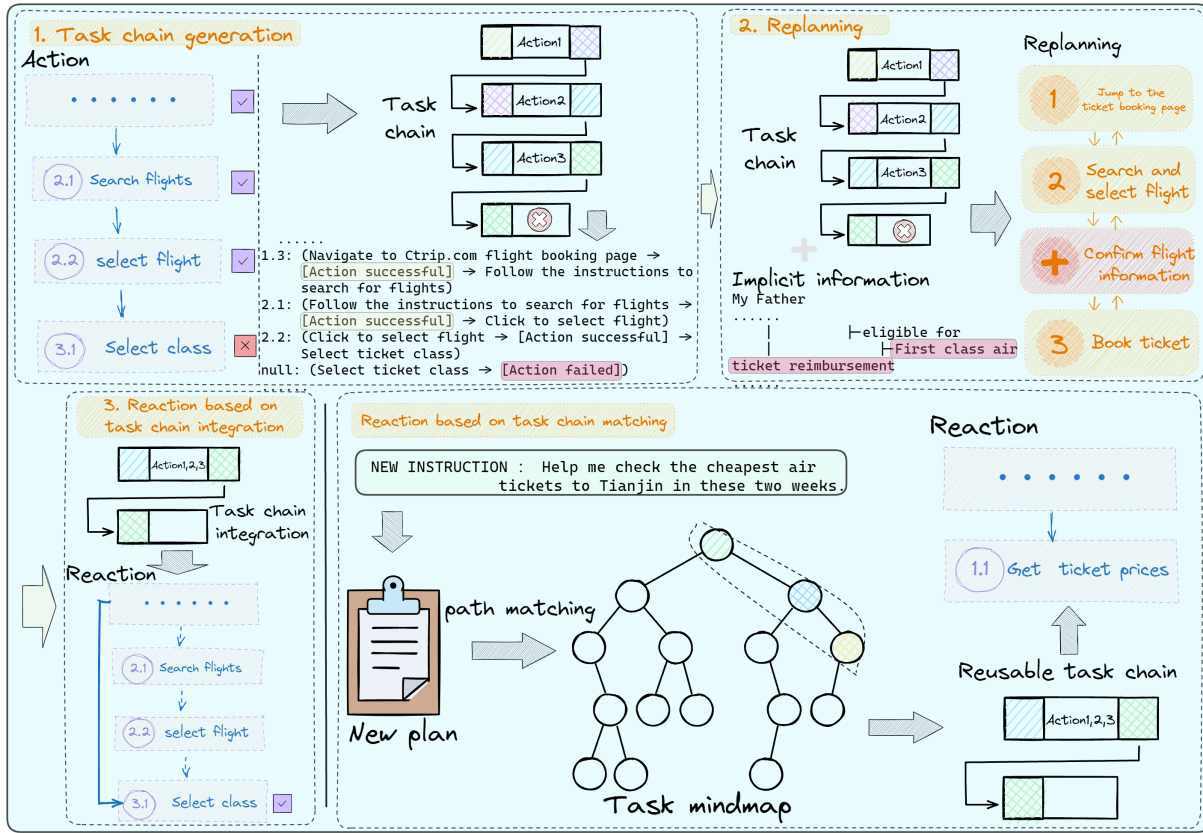


Figure 5: Task chain based Replanning and two cases of Reaction: the upper left is the generation process of the task chain; the upper right is Replanning based on task chains and implicit information; the Reaction on the lower left is used to reuse the current task; the lower right is the Reaction that reuses similar task chains.

different interaction needs of users on ticket booking service websites in the real world, we collected 812 user instructions. These 812 instructions were divided into 417 simple instructions for browsing specific pages, querying flights, comparing prices and other short task chain instructions in the A set, and 204 instructions requiring long task chains for interactions such as booking and changing tickets, which are sophisticated tasks with full information instructions in the B set. In addition, 171 instructions were based on B set, designed to simulate real human instructions with incomplete information, instruction to C set, and a knowledge base with implicit information for C set, more details on the setup can be found in the appendix.B. Based on realistic considerations, we set task execution standards for each instruction to judge whether a task was successful.

3.2 Baselines

To achieve multi-round interactions on real web pages, LLM agents need to receive feedback from the environment and call APIs multiple times to obtain reasoning about the task and generate code to interact with the environment. Not every agent method is compatible with such a complex process. Our goal is to compare the performance differences between different agent methods when

interacting with real web pages based on instructions, due to differences in reasoning and action modes. Therefore, for the sake of fairness, we have done our best to select and adapt some representative algorithms as benchmarks.

React [42] uses chain-of-thought prompts[47] to advance to the next task based on a [think, act, observe] process. We provide it with environmental feedback from our website and agent state as observations. The action stage provides executable skills generated on Ctrip.com as tools for use, and when instructions requiring implicit information are encountered, the knowledge base is passed as a pre-prompt.

Reflexion [24], which is similar to React [42] in its execution logic, includes a module that can reflect on and correct the trajectory of task completion. We recorded the trajectory of the task execution and provided it to Reflex when errors occurred so that it could correct the errors.

VOYAGER [33] has a skill library module for generating and storing executable code, and a mechanism for handling code errors based on iterative prompts. To minimise the impact of random errors in code generation, in addition to VOYAGER's own generation of executable code, we also provided VOYAGER with an executable skill library generated on Ctrip.com for comparison.

Table 1: the iteration method has been set in the baseline method during adaptation, all methods execute the task once until an error occurs or the task is completed, and the number of successful steps is recorded.

Method	$success_A steps_A$		$success_B steps_B$		$success_C steps_C$	
React	28.1%	1.9	8.3%	2.4	—	—
Reflexion	35.4%	2.3	10.9%	2.6	—	—
VOYAGER	78.2%	2.6	46.1%	5.5	8.3%	3.2
VOYAGER+Our Skill	79.4%	2.6	46.1%	5.5	10.9%	3.4
Our Method	85.8%	2.8	73.5%	6.5	55.7%	6.1

3.3 Evaluation Metrics

In tests on real web pages, some overly complex instructions could lead to poor performance across all agent methods, resulting in the inability to complete the final task. However, a higher number of successful actions for individual tasks also indicates that the method is more robust and performs better [41]. Therefore, we chose two metrics to evaluate the performance of the methods from different perspectives. The two experimental metrics are:

$success_rate_{A,B,C}$ represents the task success rate of agents under the instruction sets A, B, and C.

$$success_rate_{A,B,C} = \frac{success_task_{A,B,C}}{total_instructions_{A,B,C}} \quad (1)$$

$ave_action_steps_{A,B,C}$ represents the average number of steps that the agent takes successfully action under each instruction set.

$$ave_action_steps_{A,B,C} = \frac{action_steps_{A,B,C}}{total_instructions_{A,B,C}} \quad (2)$$

3.4 Results

We systematically evaluated the performance of our method and the baseline in completing tasks on Ctrip.com, as shown in 1. The overall experimental results show that our method outperformed the others. It achieved the highest $success_rate_{A,B,C}$ and $ave_action_steps_{A,B,C}$ across the three instruction sets.

ReAct and Reflexion currently only achieve some success with instruction set A. For instructions set B and C, it is difficult to make meaningful progress, one reason being that instructions in B and C involve multiple interactions and the use of implicit information, and these methods were originally designed for NLP tasks. Although we have provided a directly callable library of skills, the lack of components for interacting with the real world and feedback mechanisms in these methods means that they can perform some tasks on web pages, such as browsing and finding information in short task chains, but have limited interaction capabilities for real complex tasks. The current baseline models are lacking in these areas, highlighting the need for further development and research to improve their performance as agents in real-world tasks.

Voyager shows an overall performance comparable to our method on instructions set A and slightly lower on instructions set B. However, when faced with set C, there is a significant drop in performance. This can be attributed to Voyager's difficulty in using information from the knowledge base, which is too scattered and

Table 2: Replanning, Logical path reasoning, and Reaction are three progressive settings, so we start with the Replanning that has the greatest impact and remove them in order.

Method	$success_A steps_A$		$success_B steps_B$		$success_C steps_C$	
Our Method	85.8%	2.8	73.5%	6.5	55.7%	6.1
- Replanning	81.0%	2.7	66.3%	6.0	12.9%	3.8
- Logical path reasoning	83.6%	2.8	69.9%	6.5	20.7%	4.0
- Reaction	85.4%	2.8	73.0%	6.5	50.1%	6.0

fragmented, resulting in insufficient information to complete long task chains and difficult tasks.

3.5 Ablation Studies

To explore the impact of key components on our method, we sequentially removed three critical designs (Replanning, Logical path reasoning, Reaction), with detailed settings shown in the appendix and experimental results presented in the 2. We highlight the following key findings:

- The Replanning method is crucial for completing tasks in instruction set C. It helps to correct the course when the initial exploratory execution leads to a broken task chain due to an incorrectly designed plan based on insufficient information, by feeding these task trajectories back into the LLM for overall correction.
- Retaining Replanning actions, but providing discrete external knowledge bases rather than implicit information through logical path reasoning, is limited by the LLM context and struggles to accurately find the information needed to solve problems and perform correct Replanning.
- In addition to $success_rate_{A,B,C}$ and $ave_action_steps_{A,B,C}$, we compared the difference in the average number of API calls caused by removing the Reaction method. As shown in Fig. 6, Reaction ensures that successful actions from previous Replanning can be reused without having to reason from scratch. It ultimately creates a comprehensive mind map containing multiple task chains, thereby achieving reuse at the planning level. This reduces the impact of random errors in LLM-generated reasoning and code, and the correct plan can be specified from the outset for similar tasks. This reduces the impact of randomness errors from LLM-generated reasoning and code, making task execution more robust. In addition, as shown in Table 2, reaction significantly reduces the number of API calls required, thus reducing the time and token consumption required for LLM reasoning.

4 RELATED WORK

4.1 Large Language Models

In the early stages of Large Language Models (LLMs) research, the primary methods were based on transformer [32] architecture models such as BERT [12] and GPT [4, 18, 19], which are NLP (Natural Language Processing) models characterised by their reliance on text pre-training. This allows the models to effectively handle two downstream tasks: semantic representation [1] and

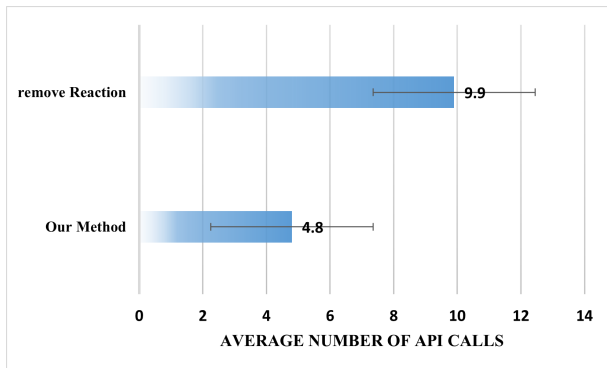


Figure 6: The reacton mechanism significantly reduces the number of API calls

text generation [6]. As the model architecture has been continuously improved, with increases in model parameters and training corpora, LLMs have demonstrated powerful emergent capabilities, capable of performing zero-shot NLP downstream tasks and some human language-based reasoning tasks [2, 16, 28, 30]. Recent LLM research, including ChatGPT [16], GPT-4 [2] and LLAMA-70B [30], has demonstrated excellent natural language generation and understanding capabilities. The coherence and grammatical accuracy of the content generated by LLMs has steadily improved, demonstrating the logic at the level of LLMs' thinking. This implies that LLMs have the potential for significant autonomy in exploration and decision making, to the extent that they can formulate viable plans based on explicit human instructions. At the same time, LLMs face significant challenges when dealing with vague instructions or interpretations with multiple meanings [27]. For humans, capturing the implied meanings in conversations comes naturally [22], but for LLMs, they are unable to formalise the implied meaning behind the input text into a reward function that allows them to select options that match the speaker's preferences in unseen contexts [29]. Therefore, our method adopts a logic path reasoning based mind map to integrate all the information. By exploiting the symbolic understanding capabilities of LLMs, it enables LLMs to perform highly personalised and accurate planning.

4.2 LLM-based Agent

Due to the impressive planning and decision-making capabilities demonstrated by Large Language Models (LLMs), LLM-based agents have been applied to a variety of real-world scenarios, such as automated robot control [3] and autonomous gaming [34]. Such an agent paradigm consists of two key components: reasoning and action. The reasoning component is primarily composed of LLMs, which are responsible for receiving input information and representing reasoning and planning processes. At the thought level, the representative Chain of Thought (CoT) [37] method has been shown to stimulate the reasoning abilities of large language models by guiding LLMs to generate reasons before producing an answer. Through planning [25], agents can break down complex tasks into more manageable subtasks and formulate appropriate plans for each, improving the likelihood of success; by designing knowledge

storage modules to store knowledge in summaries, vectors, or other data structures [43], it enables LLMs to retrieve available knowledge and generate more robust plans. At the action level, the agent approach provides LLMs with specific skills and tool manipulation capabilities that allow them to influence and shape the environment based on their thought processes and provide feedback. The React [42] framework provides a method for obtaining environmental information by observing and updating state actions, and some methods allow clues from task completion or post-action observations to help correct action errors. For example, the reflexion [24] design allows for iterative modification of action trajectories; VOYAGER [33] provides a specialized toolkit (skill library) that allows LLMs to perform actions in specific domains and correct errors through environmental feedback; some methods advocate human-agent interaction, allowing them to correct some errors and use such feedback to modify action errors. However, these reasoning-based agent methods also have their potential drawbacks. Agent-generated plans often lack awareness of common sense knowledge gained from real-world exploration and implicit information gained from deep reasoning about facts, leading to the formulation of unachievable plan [7], such as "How to build a naval fleet in Mongolia". Therefore, our method details how to transform the thought-driven action agent paradigm into a dynamic interactive process in which thought guides action, action generates new knowledge, new knowledge integrates implicit knowledge, and guides further reasoning and planning.

5 CONCLUSION

x x x x x x x x

ACKNOWLEDGMENTS

To Robert, for the bagels and explaining CMYK and color spaces.

REFERENCES

- [1] Omri Abend and Ari Rappoport. 2017. The state of the art in semantic representation. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 77–89.
- [2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [3] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. 2022. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691* (2022).
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [5] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. 2023. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712* (2023).
- [6] Asli Celikyilmaz, Elizabeth Clark, and Jianfeng Gao. 2020. Evaluation of text generation: A survey. *arXiv preprint arXiv:2006.14799* (2020).
- [7] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2023. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology* (2023).
- [8] Xiaojun Chen, Shengbin Jia, and Yang Xiang. 2020. A review: Knowledge reasoning over knowledge graph. *Expert Systems with Applications* 141 (2020), 112948.
- [9] Xuanting Chen, Junjie Ye, Can Zu, Nuo Xu, Rui Zheng, Minlong Peng, Jie Zhou, Tao Gui, Qi Zhang, and Xuanjing Huang. 2023. How Robust is GPT-3.5 to

- Predecessors? A Comprehensive Study on Language Understanding Tasks. *arXiv preprint arXiv:2303.00293* (2023).
- [10] Andrew Kean Gao. 2023. Vec2Vec: A Compact Neural Network Approach for Transforming Text Embeddings with High Fidelity. *arXiv:2306.12689 [cs.CL]*
- [11] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. 2022. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International Conference on Machine Learning*. PMLR, 9118–9147.
- [12] Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT*. 4171–4186.
- [13] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems* 35 (2022), 22199–22213.
- [14] Ruibo Liu, Ruixin Yang, Chenyan Jia, Ge Zhang, Denny Zhou, Andrew M Dai, Diyi Yang, and Soroush Vosoughi. 2023. Training Socially Aligned Language Models in Simulated Human Society. *arXiv preprint arXiv:2305.16960* (2023).
- [15] Reichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. 2021. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332* (2021).
- [16] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems* 35 (2022), 27730–27744.
- [17] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. 1–22.
- [18] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. (2018).
- [19] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
- [20] Manolis Savva, Abhishek Kadian, Oleksandr Maksymets, Yili Zhao, Erik Wijmans, Bhavana Jain, Julian Straub, Jia Liu, Vladlen Koltun, Jitendra Malik, et al. 2019. Habitat: A platform for embodied ai research. In *Proceedings of the IEEE/CVF international conference on computer vision*. 9339–9347.
- [21] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761* (2023).
- [22] John R Searle. 2007. What is language: some preliminary remarks. *Explorations in Pragmatics. Linguistic, cognitive and intercultural aspects* (2007), 7–37.
- [23] Bokui Shen, Fei Xia, Chengshu Li, Roberto Martín-Martín, Linxi Fan, Guanzhi Wang, Claudia Pérez-D'Arpino, Shyamal Buch, Sanjana Srivastava, Lyne Tchapmi, et al. 2021. iGibson 1.0: A simulation environment for interactive tasks in large realistic scenes. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 7520–7527.
- [24] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- [25] Chan Hee Song, Jiaman Wu, Clayton Washington, Brian M Sadler, Wei-Lun Chao, and Yu Su. 2023. Llm-planner: Few-shot grounded planning for embodied agents with large language models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2998–3009.
- [26] Theodore R Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L Griffiths. 2023. Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427* (2023).
- [27] Niket Tandon, Aparna S Varde, and Gerard de Melo. 2018. Commonsense knowledge in machine intelligence. *ACM SIGMOD Record* 46, 4 (2018), 49–52.
- [28] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. Alpaca: A strong, replicable instruction-following model. *Stanford Center for Research on Foundation Models*. <https://crfm.stanford.edu/2023/03/13/alpaca.html> 3, 6 (2023), 7.
- [29] Stefanie Tellex, Thomas Kollar, Steven Dickerson, Matthew Walter, Ashis Banerjee, Seth Teller, and Nicholas Roy. 2011. Understanding natural language commands for robotic navigation and mobile manipulation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 25. 1507–1514.
- [30] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [31] Robin R Vallacher and Daniel M Wegner. 1987. What do people think they're doing? Action identification and human behavior. *Psychological review* 94, 1 (1987), 3.
- [32] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [33] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291* (2023).
- [34] Zihao Wang, Shaofei Cai, Anji Liu, Xiaojian Ma, and Yitao Liang. 2023. Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents. *arXiv preprint arXiv:2302.01560* (2023).
- [35] Zhen Wang, Jianwen Zhang, Jianlin Feng, and Zheng Chen. 2014. Knowledge graph embedding by translating on hyperplanes. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 28.
- [36] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. 2022. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682* (2022).
- [37] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.
- [38] Yilin Wen, Zifeng Wang, and Jimeng Sun. 2023. Mindmap: Knowledge graph prompting sparks graph of thoughts in large language models. *arXiv preprint arXiv:2308.09729* (2023).
- [39] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. 2023. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864* (2023).
- [40] Tianbao Xie, Fan Zhou, Zhoujun Cheng, Peng Shi, Luoxuan Weng, Yitao Liu, Toh Jing Hua, Junming Zhao, Qian Liu, Che Liu, Leo Z. Liu, Yiheng Xu, Hongjin Su, Dongchan Shin, Caiming Xiong, and Tao Yu. 2023. OpenAgents: An Open Platform for Language Agents in the Wild. *arXiv:2310.10634 [cs.CL]*
- [41] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems* 35 (2022), 20744–20757.
- [42] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629* (2022).
- [43] Wanjun Zhong, Lianghong Guo, Qiqi Gao, and Yanlin Wang. 2023. MemoryBank: Enhancing Large Language Models with Long-Term Memory. *arXiv preprint arXiv:2305.10250* (2023).
- [44] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. 2023. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854* (2023).

A PROMPT DESIGN

This section contains the prompt templates for important parts.

A.1 Main prompts for Logical path reasoning

Entity extraction of instructions

You are now my personal human assistant, I'm going to give you a goal, you need to help me find the travel-related entities, for each [goal] provided below, extract and list the travel-related entities mentioned.

Example:

Input:

[goal]: I need to rush from Chengdu to Hong Kong on 2023-12-20.

Output:

['Myself', 'Chengdu', 'Hong Kong', '2023-12-20']

Warning:

1. Return entities as a list.
2. Do not repeat the input content, only return the entities list.
3. If the travel-related entity cannot be found, an empty list [] should be returned.

Mindmap integration

Please represent a text-based mindmap based on the dictionary below:

Input:

[dict]:{}

Warning:

1.The keys at the first level of the dictionary represent entity nodes. Each entity node corresponds to a knowledge graph reasoning path. entity nodes with paths should be tried to be associated to expand into a text-based mindmap.

2.It does not ask you to find the longest path, but allows you to complete the most complete logic diagram. Except for entities with empty values, do not ignore any keys.

3.No need for any text supplements or prompts.

Clean implicit information

Please combine the following goal and dictionaries to extract valuable knowledge:

Input:

[goal]:{}

[paths_dict]:{}

Warning:

1.Focus on longer paths as they may contain more useful knowledge.

2.Only focus on goal-related knowledge and ignore others.

3.Write a brief paragraph without any cliché nonsense.

A.2 Main prompts for action stage

Initial action prompt

You are now my personal human assistant, I'm going to give you a goal, in order to complete the goal with selenium python, you need to think based on observations to get the tasks that should be done at the moment, your one action should follow the framework below, loop over actions until the final goal is completed:

Input:

[observation]:Information returned from me, from the previous action and from the website page.(externally given)

Output:

[thinking]:Thinking based on the given [observation], to make a step-by-step plan to accomplish the goal,and decide the current task, return what the current task should to be done and the input required to complete the task in a dictionary like thiscurrent

task,input. If there is not enough information to support your decision about the current task and the input, ask user for more information by returningquery:the information needed, user will get back to you with more [observation], then you continue to specify the current task . based on the [observation] and previous actions, If you think the goal is done, return goal success.

Warning:

1.Reply in strict accordance with the format requirements

2.Before your output, always check if there is enough information to support you to RETURN the [current task], if you can't specify the current feasible task and the input, ask the user for more information, don't give free rein.

3.After [observation] of the current task completion will be returned, in the [thinking], if the final goal is judged to be complete then only return the goal success.

Web coder

You are now a useful automated web coder,I'm going to give you a task, your your code generates should follow the framework below(step1,step2):

step1:

Input:

[task]: current task should to be done.(externally given)

[text]: html text of current web page, which contains all the text of the current page, and each text corresponds to an operable element of the web page, this is the information of the page you observed.(externally given)

Output:

[selector]: Select the text element or text elements most relevant to the [task] at hand.

step2:

Input:

[html]:The web page html corresponding to the text element selected by the [selector].(externally given)

Output:

[code]:Use selenium python to find the corresponding element in [html], generate concrete and feasible code to perform operations.

Warning:

1.The content of input is input from the outside world, you need to answer the content of output according to the context.

2. Just return the results that need to be returned, do not reason or describe on this basis.

3. The generated code should be clearly executable based on the given html, do not generate sample code that cannot be demonstrated.

Replanner

There is a problem with the plan you made earlier. I will give you the following input and ask you to break away from the limitations of the original information and re-formulate a feasible plan:

Input:

[goal]: {}

[pre_plan]: {}

[task chain]: Here is a series of actions taken to complete the task. {}

[implicit information]: {}

Warning:

1. Use the action records and feedback of the task chain, combined with implicit information to fix errors that cause the goal to be unable to be completed.

2. If information is still missing to resolve the problem, abort and request a re-invocation of more KGs to obtain the implicit information.

3. It is not only necessary to fix the errors at the breakpoints of the task chain, but also to improve the plan from an overall perspective.

B EXAMPLE OF INSTRUCTIONS

This section contains examples of instruction sets.

A: simple instructions

- (1) Select One way on the ticket page of booking.com.
 - (2) Find the cheapest flights from Chengdu on Ctrip.com.
 - (3) Set the departure point as Chengdu and the destination as San Francisco on Ctrip.com.
-

B: instructions requiring long task chains

- (1) Help me book the most expensive first class ticket from Paris to London on January 1, 2024 on Booking.com.
 - (2) Help me log in to my Booking account and check the baggage requirements for the latest flight of my order.
 - (3) Check if there is a flight from Chengdu Shuangliu International Airport to Tianjin tomorrow, October 30, 2023 for less than 600 CNY on Ctrip.com.
-

C: real human instructions with incomplete information

- (1) Help my wife immediately change her ticket to the nearest flight home.
 - (2) Help me book a flight ticket to Hong Kong to attend the academic conference on Ctrip.com as soon as possible.
 - (3) As usual, help my father arrange round-trip air tickets for this weekend's trip.
-

Examples of KG triple knowledge base of some simple basic facts for C:

-
- [My father, dislikes traveling by, boat]
 - [My mother, prefers accommodation, boutique hotels]
 - [My wife, enjoys, mountain hiking]
 - [Myself, often book, cheapest flights]
 - [My mother, is afraid of, bungee jumping]
 - [My wife, always carries, travel journal]
 - [My wife, aspire to travel to, New Zealand]
 - [My father, is a, Professor] [Academic conference, is on date, August 20]
 - [Academic conference, is in location, Hong Kong]
 - [Professor, requires attendance, Academic conference]
 - [Academic conference, eligible for, First class air ticket reimbursement]
 - [Myself, are in location, Chengdu]
 - [My wife, are in location, Tianjin]
 - [My trip to Tianjin, is for, weekend date]
 - [Weekend date, will choose, cheapest air ticket]
 - [Cheapest air ticket, is, Economy class air tickets]
 - [Weekend date, will choose, cheapest air ticket]
 - [My father, is in location, Chengdu]
 - [Hong Kong, need, HK/MACAU TRAVEL PERMIT]
-

C CASES

This section shows cases of interacting with real-world web pages.

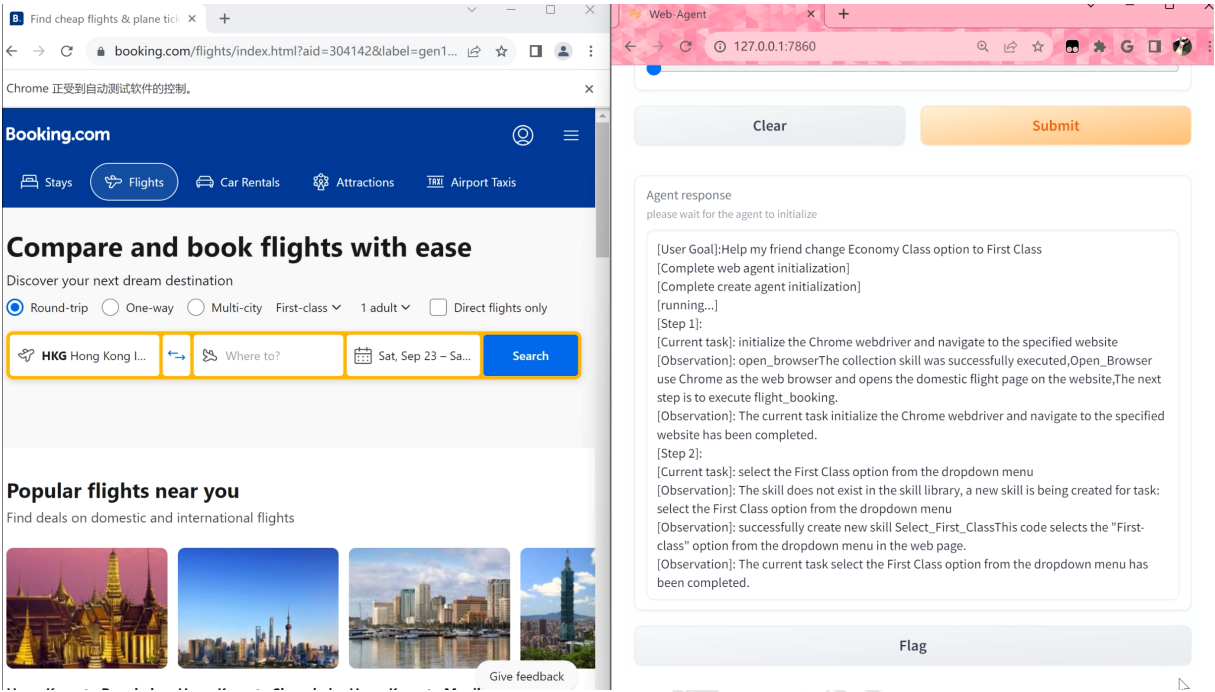


Figure 7: A real case of using our method to interact with the Booking.com

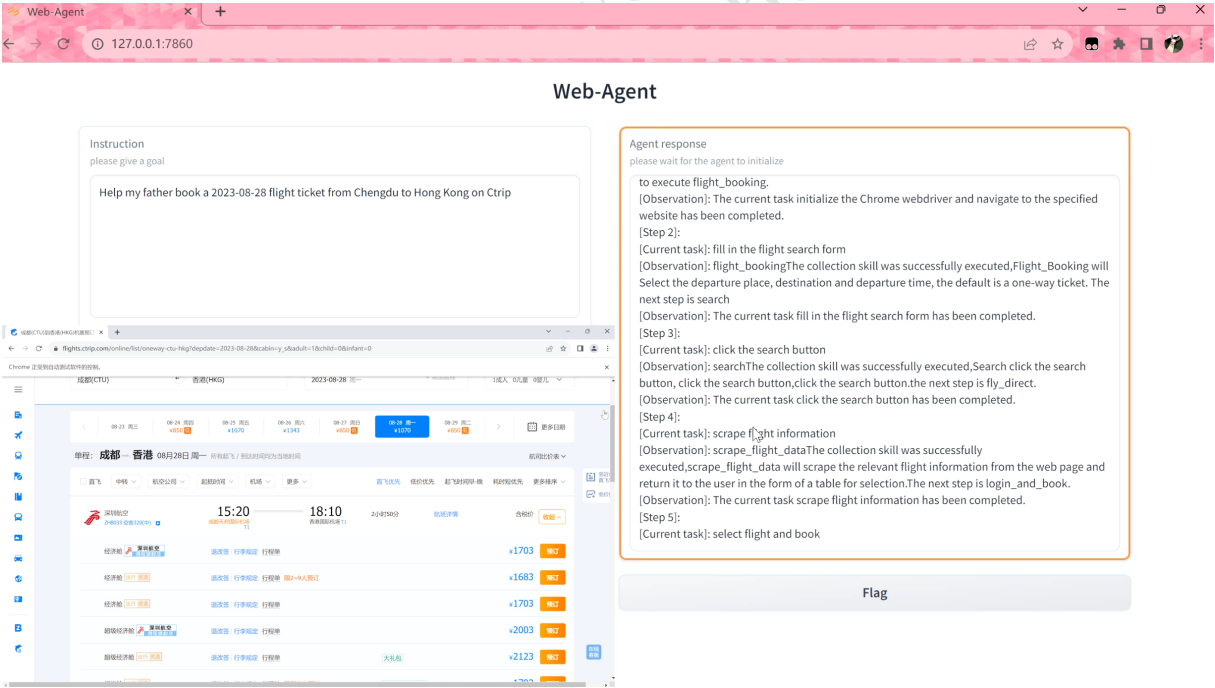


Figure 8: A real case of using our method to book tickets on Ctrip.com.